

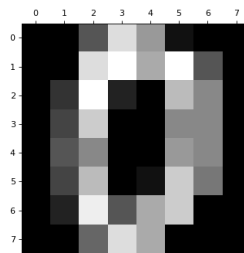
Assignment 6 Mini Project

5. Chosen Task: Machine learning sustainability estimator: Implement a solution that given learns the energy efficiency of different ML models, and given a task uses the most energy efficient ML model suitable for the task. At the first stage, only report the ML performance, energy consumption and carbon emissions for all models. At the second stage, select the model that will best balance ML performance and emissions, considering also carbon intensity.

Goal: Implement a family of DNN with varying number of layers and neurons per layer and measure the energy consumption using turbostat to choose a set of configs for a given energy consumption and accuracy requirement.

Dataset Used:

- Scikit load_digits
- https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html
- Each datapoint is an 8x8 image of a digit as seen below.

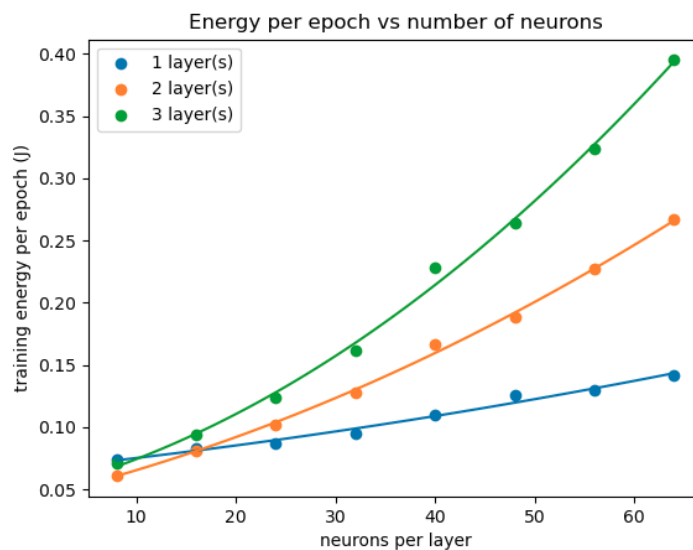


Procedure: To implement the goal, I created 2 files loop.py and trainer.py. To run it download both files and run in the terminal:

```
python3 loop.py
```

The orchestrator file “loop.py” runs the “trainer.py” file with different arguments each time for the neuron and layer count, as well as a seed to make the results reproducible. The energy is measured in joules using turbostat and averaged. The results for the training can be seen as follows:

layers	neurons	accuracy	energy(J)	energy per epoch	gCO2	time(s)
1	8	0.954 +/- 0.007	16.27 +/- 0.16	0.0542	0.0009	1.4714
1	16	0.964 +/- 0.004	23.17 +/- 3.49	0.0772	0.0013	1.7991
1	24	0.969 +/- 0.004	23.64 +/- 0.17	0.0788	0.0013	2.0888
1	32	0.968 +/- 0.004	27.56 +/- 1.1	0.0919	0.0015	2.3739
1	40	0.974 +/- 0.003	34.09 +/- 1.68	0.1136	0.0019	2.9261
1	48	0.971 +/- 0.001	34.16 +/- 0.29	0.1139	0.0019	2.9939
1	56	0.976 +/- 0.003	37.91 +/- 0.26	0.1264	0.0021	3.2891
1	64	0.976 +/- 0.002	42.19 +/- 1.09	0.1406	0.0023	3.5846
2	8	0.945 +/- 0.005	18.35 +/- 0.09	0.0612	0.001	1.6569
2	16	0.964 +/- 0.004	24.54 +/- 1.46	0.0818	0.0014	2.1192
2	24	0.969 +/- 0.004	30.35 +/- 0.1	0.1012	0.0017	2.6528
2	32	0.971 +/- 0.002	38.47 +/- 0.58	0.1282	0.0021	3.2801
2	40	0.972 +/- 0.003	51.03 +/- 0.98	0.1701	0.0028	4.43
2	48	0.972 +/- 0.006	56.74 +/- 0.24	0.1891	0.0032	4.8851
2	56	0.975 +/- 0.004	68.37 +/- 0.94	0.2279	0.0038	5.8075
2	64	0.975 +/- 0.001	80.74 +/- 0.3	0.2691	0.0045	6.8157
3	8	0.919 +/- 0.017	20.81 +/- 0.18	0.0694	0.0012	1.83
3	16	0.96 +/- 0.006	28.5 +/- 1.19	0.095	0.0016	2.4486
3	24	0.963 +/- 0.009	37.47 +/- 0.15	0.1249	0.0021	3.223
3	32	0.964 +/- 0.005	48.62 +/- 0.26	0.1621	0.0027	4.1799
3	40	0.969 +/- 0.002	68.87 +/- 1.21	0.2296	0.0038	5.9157
3	48	0.969 +/- 0.004	79.36 +/- 1.97	0.2645	0.0044	6.7554
3	56	0.976 +/- 0.003	97.38 +/- 0.85	0.3246	0.0054	8.3237
3	64	0.975 +/- 0.001	119.67 +/- 1.37	0.3989	0.0066	10.0242



Analysis:

It can be seen that accuracy saturates around 0.97 across almost every config, while energy cost varies by more than 6x, so energy efficiency can be seen as the deciding factor. Increasing the width gets diminishing returns past about 24 neurons: a single-layer net jumps from 0.954 at 8 neurons to 0.969 at 24, then saturates to only 0.976 at 64 neurons despite doubling its energy (26.1 J to 42.6 J). Depth is counterproductive here. Adding layers gives no accuracy benefit, as the best 1-, 2-, and 3-layer models all land at 0.975 to 0.976 (they are within the error bars of each other, so this can be seen as a “limit” of DNN for this task with small number of layers), but increases energy sharply, since the 3-layer/64-neuron model burns 118.5 J for the same result the 1-layer/56-neuron model achieves at 38.9 J, roughly a 3x penalty for no increase in accuracy.

The worst performance in terms of accuracy was the three layers, 8 neuron config. This can be explained by how DNN's train. Through backpropagation, the error signal gets sent back through the layers. As the number of layers increases, the signal gets weaker. If one neuron is "stuck", which happens randomly, then the accuracy would be pushed down. For a 64 neuron DNN, 1 neuron being stuck doesn't affect it as much.

The optimal choice of layer and neurons were calculated using the "Pareto frontier", which calculates if any one config is strictly better than, or dominating, the rest. It also ranks based on accuracy and energy consumption if there is no dominating config.

```
pareto_front = []
for r in results:
    is_dominated = False
    for other in results:
        more_accurate = other["accuracy"] >= r["accuracy"]
        less_energy = other["energy"] <= r["energy"]
        strictly_better = (other["accuracy"] > r["accuracy"]) or (other["energy"] < r["energy"])
        if more_accurate and less_energy and strictly_better:
            is_dominated = True
            break
    if not is_dominated:
        pareto_front.append(r)
```

Pareto front (best trade-offs):

- 1 layers, 8 neurons
- 1 layers, 24 neurons
- 1 layers, 40 neurons
- 1 layers, 56 neurons
- 2 layers, 8 neurons
- 2 layers, 16 neurons

```
Best choice: 1 layers, 16 neurons
1 layer(s): a = -1.635044642857154e-05  b = 0.002814335317460325  c = 0.02805285714285691
2 layer(s): a = 2.486917162698406e-05  b = 0.0017718898809523825  c = 0.04841130952380945
3 layer(s): a = 5.332651289682538e-05  b = 0.001982544642857139  c = 0.04817535714285721
```

saved energy_vs_neurons.png

After calculating the Pareto frontier, the optimal region is 1-layer at 24 to 40 neurons, which gives about 0.969 to 0.974 for 26 to 33 J, or 2-layer at 8-16 neurons. Because gCO₂ is proportional to energy, every conclusion about joules carries over to carbon emissions. The conclusion is that for this task the accuracy above 0.97 is not worth the extra energy consumption and the cheapest acceptable model dominates the more elaborate ones.

```
pick acc>=0.96, <=15J: none fit
```

A search mechanism for a model that satisfies prior needs is also implemented. The required accuracy and energy requirements can be changed in the start of the code and at the end the most efficient model config that meets the requirements is returned.

This projects covers the concepts taught in the CWM in the following ways:

- Performance:
 - The focus of this research is energy usage/energy efficiency, one of the second order performance metrics. Energy usage is compared with the accuracy in order to get an answer to the question: what would be the most optimal energy wise for a given accuracy?
 - Scalability: Anyone can easily scale the results for a larger number of layers and neurons by changing the variable values.
- Measurement:
 - All energy measurements were done across 5 seeds and averaged, while also calculating the standard deviation for uncertainty calculation. This was done to get statistically significant results.
 - Time measurements were conducted for each run to see how long this simple training for 300 epochs took.
- Sustainability:
 - In this simple model, the carbon footprint of the DNN is directly proportional to the energy consumption. This means that carbon footprint was a redundant information and didn't contribute to the most optimal model choice. However, the CO2 emissions for all training sessions were calculated and tabulated in the results.
- Cyber Threats:
 - Due to limited scope project, cyber security wasn't thoroughly covered.

Use of AI:

- Turbostat logic help:
 - <https://claude.ai/share/3c9c820e-f593-408d-ae94-962949282f63>
- “Pareto frontier” logic help
 - <https://claude.ai/share/acb3a9af-8b79-4815-8c79-d21c695e0f8a>